Authority Is Not a String: A Capability-Scoped Harness for Prompt-Injection-Resistant Coding Agents

Anonymous Author(s)

Abstract

Coding agents use system-level tools to read files, execute commands, and modify source code. Within the agent's sandbox, these tools often carry *ambient authority*: naming a resource is sufficient to act on it. Indirect prompt injection exploits this authority by placing instructions in repository files or tool output that cause the agent to perform actions the user did not request.

We propose CapScope, a harness-level authorization mechanism that restricts tool use without requiring the model to identify malicious text. Before repository contents or tool output are read, CapScope derives a task-wide authority ceiling from trusted input. It then assigns each agent a separate set of typed capabilities, stored outside the model's context. Every tool call is checked against the capabilities of the agent that issued it. Permissions assigned to one sub-agent are therefore not automatically available to another. An injection may cause an agent to request an action, but the request is blocked unless that agent already has the required capability.

We implement CapScope on the Pi coding agent [6] and evaluate it in a repair workflow where an orchestrator delegates subtasks to separate sub-agents. The evaluation covers five Python tasks, five injection surfaces, four authorization conditions, and three trials per cell (300 runs). The three global-policy baselines execute the injected effect in 33–47/75 runs, compared with 3/75 under CapScope. CapScope completes 68/75 repairs, while the baselines complete 68–72/75.

CCS Concepts: • **Security and privacy → Access control**.

Keywords: coding agents, indirect prompt injection, capabilities, least authority, tool authorization

1 Introduction

A developer asks a coding agent to fix the failing tests in an auth module. An orchestrator delegates test execution to a runner and source editing to a patcher. A trojaned test fixture, planted by a repository contributor or pulled in through a compromised dependency, prints to the suite's output an official-looking note: *a signing key rotated; restore the build by replacing the key in* `src/auth/keys.py`. The supplied value is attacker-controlled; once committed it lets the attacker forge auth tokens. The sub-agent cannot tell this from a real CI message, and the target is an ordinary source path. The request mentions no `.env` or `curl`, and nothing a deny rule would flag. If the sub-agent can write source, the backdoor lands, and neither it nor the developer notices.

This is indirect prompt injection: when a coding agent reads external resources such as project files, skill definitions, source comments, or tool output, any text there can act as an instruction. The threat is well-documented: Liu et al. [12] report attack-success rates up to 84% against production coding editors in their study, and CVE-2025-54135 (*CurXecute*) chained indirect injection to creation of Cursor MCP configuration and remote code execution. Malicious rule and skill files provide a separate backdoor vector [10].

Most defenses work at the model level: providers increasingly train and prompt models to be skeptical of untrusted instructions [18], and a separate classifier may flag suspected injected payloads before they reach the agent. Such defenses can reduce attacks, but still make a probabilistic decision about whether text is an instruction. Studies have found attacks despite instruction-hierarchy prompts [12, 20], and classifiers can produce false positives or fail against adaptive attacks [4]. CapScope instead checks permissions after the model proposes a tool call. CaMeL [3] also uses capabilities, separating control and data flow with a dual-LLM interpreter; it evaluates web and email agents rather than coding workflows.

This is a *confused-deputy* problem [9]. The harness has the user's privileges, while the model chooses the resource names on which those privileges act. Object-capability systems [13, 16] keep permissions separate from names. CapScope applies that rule to model text and tool calls.

Our approach: CapScope. CapScope fixes the task's maximum authority from trusted input and then gives each agent only the part of it that agent's step requires. It first makes a separate LLM call using only the trusted user request and the project's file tree, which predicts the files and commands the task will need. The host freezes these permissions—stored outside the model's context, so text encountered later cannot expand them—before the agent reads the repository or runs a command.

When an agent delegates work, the sub-agent receives only the permissions its subtask needs, bounded by the delegating ceiling. Every tool call is then checked against the permissions of the agent making it. This per-agent split is what a single global policy cannot express. In the opening example, the sub-agent that reads the poisoned test output can run tests but cannot edit source: the source-write operation a patcher would be allowed to make is unavailable to the

LMPL '26, Oakland, CA, USA
2026.

runner that received the injection, so the injected key-write is blocked.

This paper contributes:

- We identify a granularity gap in current coding-agent authorization: a single task-level policy cannot give two agents different authority. We close it with a capability-scoped authorization model (Section 3).
- We implement this model in Pi as a harness-level enforcement layer: authority is minted from trusted input, attenuated per sub-agent, and checked before every tool call (Section 3.3).
- We evaluate it across five repair tasks, five injection surfaces, three baselines, and CapScope (300 runs), measuring both injected-effect execution and autonomous task success (Section 4).

Anonymous artifact. An anonymized package containing the code, tasks, mutators, harness, and decision logs is available at https://figshare.com/s/86184ed20f66d1f0cf91.

2 Background

The agent loop. A coding agent combines a model M, tools $\mathcal{T}$, and a harness $\mathcal{H}$ that runs the loop. The model reads the current *context ctx* (the message transcript) and emits a *tool proposal*: a tool name with concrete arguments, written $(t, \mathbf{a})$. The harness executes it and appends the resulting observation o to the context. Each tool has an *effect type* $\mathit{eff}(t)$ naming the kind of action it causes; we group these into Read, Write, and Exec. The harness is what actually performs effects, so it is the natural place to govern them.

Trust boundary. The values in the context do not all deserve equal trust, and the distinction is what the whole design turns on. In a session initiated by a developer, the *trusted* inputs are the user's request, the developer-configured system prompt, and the user's private settings. Everything whose provenance is the working repository or command output is *untrusted*: file contents read by the agent, stdout/stderr of executed commands, and project-local config files any repository contributor can modify.

Because the model reads both trusted and untrusted text, its tool calls are treated as untrusted requests [12]. The harness does not infer authority from a call; it checks authority stored outside the conversation.

2.1 Coding Agents

CapScope targets coding agents whose harness mediates tool calls and maintains a separate capability store for each agent session, which we call a *principal*.

Tools and effects. A coding agent's tools cause three kinds of effect. Read and Write concern the filesystem. Exec—running a shell command—is the broadest: an unrestricted command can reach the network, read secrets, rewrite git history, or install packages, all with the user's privileges. Absent a command-level restriction, Exec is the effect a misdirected agent is most often steered toward.

Injection surfaces. A coding agent loads context from several sources in the working directory. All are *Untrusted* under the trust boundary above:

(1) **README.md.** Project documentation the agent reads on demand; any repository contributor can modify it.
(2) **AGENTS.md.** A repository-controlled instruction file loaded automatically for the agent.
(3) **Skill files.** Project-local markdown prompt templates. Analogous to coding rule files, a known backdoor vector in production editors [10].
(4) **Source file contents.** An instruction in a comment or docstring enters the context as *Untrusted* data when the model reads the file.
(5) **Tool output.** The standard output and error of shell invocations return to the model as *Tool*-role messages; a trojanized test runner or network-fetched script can inject through this channel.

These surfaces define the attack scenarios in our evaluation (Section 4). Under ambient authority, each can redirect the model toward an attacker-chosen tool call; how a capability discipline changes that outcome is the subject of Section 3.

What CapScope requires of a host. A host needs a blocking pre-dispatch interceptor and per-session state. A lifecycle hook that only observes tool calls is insufficient: the interceptor must reject a call before execution. The capability representation itself is independent of the host API.

2.2 Threat Model and Non-Goals

We consider indirect prompt injection through any text that enters the model's context from repository files, project-instruction files, skill definitions, source comments, or tool output. The attacker may steer the model's proposed tool calls but cannot modify the harness, the capability store, or the tool wrappers, and cannot cause a capability to be minted. Within this model, an external effect runs only if a held capability covers it. CapScope does not stop the model from following an injected instruction; it stops the resulting tool call when that agent lacks the required capability.

CapScope is not a confidentiality mechanism. A permitted read followed by a permitted write can still move data, and an injection can misuse authority deliberately granted to its reader. Those cases require complementary information-flow controls or sandboxing.

Assumptions. The guarantee assumes a small trusted computing base: every effect is mediated; stores are host-side and isolated by principal; path and command predicates faithfully implement their structured scopes; and only the host may mint or complete a derivation. The model may propose the initial set or request a sub-agent scope, but the

host controls store updates and checks that each requested scope lies within the delegating ceiling.

The preflight reads directory and file names, but not file contents. Filename injection is outside this evaluation. The host validates and freezes the generated ceiling before any evaluated injection surface is read. The guarantee assumes correct predicates, complete mediation, and no repository contents in the preflight input.

Scope of the claim. CapScope bounds which effects a principal may invoke; it does not make every covered effect safe. An injection can misuse authority deliberately granted to its reader. An allowed command can also execute untrusted repository code transitively—`pytest`, for example, imports project modules—so process sandboxing remains necessary. These are complementary boundaries, not properties of capability lookup.

3 CapScope: The Design Pattern

CapScope replaces ambient authority with permissions held by the harness. It checks whether an installed capability covers a proposed tool call, without asking the model to classify the proposal's provenance.

3.1 Capabilities as Typed Authority

Definition 1 (Capability). *A* capability *is a pair* $c = (e, \varphi)$ *written* $Cap[e, \varphi]$, *where* $e \in$ *Effects is an* effect type *and* $\varphi : Args \to \mathbb{B}$ *is a* scope predicate *over the arguments of that effect. A capability* $Cap[e, \varphi]$ covers *a proposal* $(t, \mathbf{a})$ *when* $eff(t) = e$ *and* $\varphi(\mathbf{a})$ *holds.*

A *capability store* Σ is a finite set of capabilities held by the harness for the duration of a session. The model never holds capabilities directly; it sees only the effects they permit.

Effect types and scope predicates for coding agents. For the four tools of a typical coding agent, we define three effect types and give representative scope predicates. Here p denotes a file-system path and s a shell command string.

Effect type (tools)	Scope predicate φ
`Read` (`read`)	$\varphi(p) \triangleq within(canon(p), \pi)$
`Write` (`write`, `edit`)	$\varphi(p, \_) \triangleq within(canon(p), \pi)$
`Exec` (`bash`)	$\varphi(s) \triangleq allowed_A(parse(s))$

π: canonicalized path prefix; A: allowed argument-vector prefixes; *canon*: resolves `..` and symlinks.

$within(x, \pi)$ means that path x equals or lies below prefix π. We write `src/**` as shorthand for the canonical subtree rooted at `src/`. The predicate uses a canonicalized path (*canon* resolves `..` and symlinks) to prevent traversal attacks. $parse(s)$ rejects substitution, splits compound commands, removes redirections, and returns each segment's argument vector. $allowed_A$ holds when every vector is read/navigation or begins with a vector in A. Thus `Exec[pytest]` admits `pytest -v`. A compound command containing an uncovered `curl` segment is rejected. Redirections and transitive effects of allowed commands remain inside the process-sandbox boundary.

3.2 Formal Semantics

Notation. e is an effect type, t a tool, $\mathbf{a}$ its arguments, and o an observation. $eff(t)$ gives the effect of t; $\varphi : Args \to \mathbb{B}$ accepts or rejects arguments, where $\mathbb{B} = \{true, false\}$. $Cap[e, \varphi]$ pairs an effect with a scope; Σ_i is agent i's finite capability store; and $ctx{\cdot}o$ appends o to the context. For the root orchestrator, D_i is the task-wide ceiling; for a non-root agent, it is that agent's already-derived store. Here r is the root and $\langle X; ctx\rangle$ is a state with authority X and context ctx. Symbols $\triangleq$, $\_$, $\varnothing$, $\subseteq$, and $|$ mean "defined as," ignored argument, empty set, subset, and "such that." We write $c' \preceq c$ when c' and c have the same effect type and the scope of c' is no broader: for $c' = Cap[e, \varphi']$ and $c = Cap[e, \varphi]$, $\forall \mathbf{a}.\, \varphi'(\mathbf{a}) \Rightarrow \varphi(\mathbf{a})$. The remaining set and logic symbols $\in$, $\cup$, $\forall$, $\exists$, and $\Rightarrow$ have their usual meanings. $D_i \vdash C' \Downarrow \Sigma_{sub} = C''$ means that candidate set C' is checked under delegation ceiling D_i, and the covered subset C'' is installed as the sub-agent's capability store. $\longrightarrow$ updates an authority set, $\Downarrow$ installs a derived store or returns an observation, and $\xrightarrow{(t,\mathbf{a})}$ records an invoked tool call.

Minting: where authority comes from. Mint is the only rule that introduces authority not already bounded by a held capability. It runs a trusted host policy P and adds the finite capability set C that it returns for trusted preflight input I_T:

$$\frac{P \in Policies_{Trusted} \quad P(I_T) = C}{\langle D_r; ctx\rangle \longrightarrow \langle D_r \cup C; ctx\rangle} \text{ (Mint)}$$

With fresh $D_r = \varnothing$, this sets $D_r = C$. Host-side compilation installs $\Sigma_r \subseteq D_r$ for root calls, as in Listing 2. In our implementation, an LLM proposes capability descriptions from the trusted request and the project's file tree—names only, never contents. Deterministic host code validates and compiles the proposal as policy P before file contents, tool output, or another injection surface is available. The policy is then frozen for that session. The ceiling and operational store are fresh per session and do not accumulate across tasks. If a later call is genuinely necessary but uncovered, the harness may ask the user to approve a single additional capability. That out-of-band decision is a new trusted Mint, followed by host-side compilation; the model cannot approve its own request. Interactive approval is disabled in our evaluation.

Derivation: narrowing for delegation. A model supplies a requested grant, and host-side completion may add role-required candidate descriptions. Let C' be the resulting candidate set; its descriptions carry no authority until installed by the host. Derive filters C' against D_i, installs the covered subset C'', and drops uncovered requests:

$$\frac{C'' = \{c' \in C' \mid \exists c \in D_i.\ c' \preceq c\}}{D_i \vdash C' \Downarrow \Sigma_{sub} = C''} \text{ (Derive)}$$

For example `src/**` can be narrowed to `src/parser.py`, never the reverse. A spawned sub-agent's fresh store contains C'' alone.

Invocation: the check before every effect. INVOKE is where a proposal becomes an effect. It fires only when some held capability matches the tool's effect type and its predicate accepts the arguments; $execute(t, \mathbf{a}) \Downarrow o$ runs the tool and yields the observation o appended to the context:

$$\frac{\mathrm{Cap}[e, \varphi] \in \Sigma \quad eff(t) = e \quad \varphi(\mathbf{a}) \qquad execute(t, \mathbf{a}) \Downarrow o}{\langle \Sigma; ctx \rangle \xrightarrow{(t,\mathbf{a})} \langle \Sigma; ctx \cdot o \rangle} \ (\textsc{Invoke})$$

If no capability covers the proposal it is blocked and the model receives a refusal. Since the model's output is untrusted (the trust boundary, Section 2), the check never consults the model about provenance. The only question is whether Σ already holds a covering capability.

What the rules give us. By inspection of the three rules, every effect that executes is covered by a capability whose authority traces back to a trusted MINT. INVOKE is the only rule that runs an effect, and it requires a covering capability already in Σ. Root authority enters Σ_r only through compilation from D_r; sub-agent authority enters through DERIVE. Both can only narrow, so no agent can exceed the initial minted authority. In our evaluation, this mint occurs before untrusted content is admitted and later user-approved minting is disabled. Injected text may influence a proposed tool call or requested sub-agent grant, but it cannot expand the task-wide authority. A call outside the resulting scope is refused.

3.3 Implementation

We realize CapScope on Pi [6], an open-source TypeScript coding-agent toolkit with a CLI and a programmable SDK, without modifying Pi internals. The experiment enables Pi's four built-in tools below; the command wrapper also recognizes read/navigation programs such as `grep`, `find`, and `ls`.

Tool	Arguments	Effect
`read`	`path : string`	Read
`write`	`path : string, content : string`	Write
`edit`	`path : string, edits : Edits`	Write
`bash`	`command : string`	Exec

Pi 0.78.0 supplies the two host requirements of Section 2.1. Extensions register handlers with `pi.on`. The `tool_call` event occurs before dispatch; returning `block: true` aborts the call and sends the reason to the model. CapScope installs this hook separately for the orchestrator and each sub-agent. The SDK's `createAgentSession` returns `{session}`; callers submit work with `session.prompt()`. The `pi-subagents` extension[1] delegates a subtask to a *process-isolated* sub-agent, whose separate address space lets CapScope seed its capability store independently.

Component 1: structured capabilities. Scopes are structured data rather than opaque functions. The host can check whether a requested path prefix or command set lies within the delegating ceiling before installing it.

Listing 1. Schematic pseudocode for the structured capability type.

```
type EffectType = "read" | "write" | "exec";

type ScopeSpec =
  | { kind: "path"; prefix: string }
  | { kind: "argv"; allowed: string[][] };

interface Capability {
  effect: EffectType;
  spec: ScopeSpec;
  description: string;
}

type CapSet = Capability[];
```

Component 2: initial minting. Before loading project context, `authorize` makes a preflight LLM call using the trusted request and the project's file tree. The call predicts the permissions required by the complete task and by the orchestrator. The host validates and compiles this prediction into a task-wide *ceiling*, from which sub-agents may derive authority, and the orchestrator's operational store. The latter may be read-only. The root orchestrator is the exception to ordinary delegation: its calls use the operational store, while its grants are bounded by the task ceiling.

Listing 2. Schematic pseudocode for initial authorization.

```
interface Authority {
  ceiling: CapSet;
  orchestrator: CapSet;
}

async function authorize(
  trustedRequest: string,
  taskTree: TaskTree
): Promise<Authority> {
  const proposal = await preflightLLM({
    trustedRequest,
    taskTree
  });
  return validateAndCompile(proposal);
}
```

The `authorize` call appears once, at session start. Repository text and tool output are unavailable at this point. The preflight prediction may still be too broad or too narrow, but

[1] https://www.npmjs.com/package/pi-subagents

later injected content cannot change it. A runtime user approval, if enabled, is a separate trusted mint and is recorded as such.

Component 3: derivation and transfer. Delegation is driven by the model, just as the initial mint is. When an agent delegates, it issues a `subagent` tool call whose structured arguments carry the sub-agent's role, subtask, and a *requested grant*; the same decision that chooses what to delegate also proposes how much authority the sub-agent should get. The host treats that request as a proposal only: `derive` keeps just the capabilities structurally contained in the delegating ceiling (rule Derive) and drops the rest. The implementation first calls `ensureUsable` to supply omitted repository-local reads and role-required test or write scopes, then calls `derive` to filter the completed request against the frozen ceiling. Repository reads are allowed by default in all conditions. Because the subtask text is model-generated, it may affect which in-ceiling scope is selected, as discussed in Section 5.

Listing 3. Schematic pseudocode for deriving and transferring a sub-agent store.

```
function derive(ceiling: CapSet, requested: CapSet)
    {
  const derived = requested.filter(cap =>
    ceiling.some(p =>
      p.effect === cap.effect && scopeWithin(cap.
          spec, p.spec)
    )
  );
  return { derived };
}

async function spawnScoped(
  ceiling: CapSet, role: Role, task: string,
      requested: CapSet
) {
  const completed = ensureUsable(role, task,
      requested, ceiling);
  const { derived: subCaps } = derive(ceiling,
      completed);
  const { session: sub } = await createAgentSession
      ({
    resourceLoader: scopedRoleResources(role,
        subCaps)
  });
  await sub.prompt(task);
}
```

Component 4: per-principal dispatch checks. Every orchestrator and sub-agent session installs the same small handler, but closes over a different store. It compiles the structured scope, checks Invoke, and blocks uncovered calls.

Listing 4. Schematic pseudocode for the per-principal CapScope hook.

```
function installCapScope(pi: ExtensionAPI, caps:
    CapSet) {
  pi.on("tool_call", (event) => {
    const granted = caps.find(
      c => effectOf(event.toolName) === c.effect
        && compileScope(c.spec)(event.input)
    );
    if (!granted) {
      return {
        block: true,
        reason: `[CapScope] uncovered ${event.
            toolName} call`
      };
    }
  });
}
```

Usage: a complete session. The root's calls are checked against `auth.orchestrator`; delegation is checked against `auth.ceiling`. Thus a read-only orchestrator may grant a task-required write to a patcher, but no grant can exceed the ceiling.

Listing 5. Schematic pseudocode for orchestrator setup.

```
const auth = await authorize(userRequest, taskTree);

const { session: orchestrator } = await
    createAgentSession({
  resourceLoader: capScopeResources(auth)
});
await orchestrator.prompt(userRequest);
```

Other frameworks. Porting CapScope requires a host-specific blocking pre-dispatch interceptor and per-session state. In-process sub-agents need separate stores; separate processes receive a derived store at startup. The hook registration and session setup must be checked against each framework's API.

3.4 Example: A Backdoor the Allowlist Cannot Stop

A developer asks an orchestrator to fix the failing auth-module tests. The task needs project reads, source writes, and test commands. Before reading project content, preflight receives only the trusted request and file tree; the host compiles c_1 for reads, c_2 for writes, and c_3 for execution. Write A_{task} for the allowed prefixes `pytest` and `mypy src/`, and A_{run} for `pytest` alone.

$$c_1 = \mathrm{Cap}[\mathrm{Read},\ \mathit{within}(\mathit{canon}(p), \texttt{"project/**"})] \quad (1)$$

$$c_2 = \mathrm{Cap}[\mathrm{Write},\ \mathit{within}(\mathit{canon}(p), \texttt{"src/**"})] \quad (2)$$

$$c_3 = \mathrm{Cap}[\mathrm{Exec},\ \mathit{allowed}_{A_{task}}(\mathit{parse}(s))] \quad (3)$$

The orchestrator delegates the noisy test run to a *runner* and later the source edit to a *patcher*. The runner needs to read tests and run `pytest` but needs no write, so before spawning

Table 1. Repair tasks used in the evaluation.

Task	Bug
T1	Off-by-one error in `daterange.py`
T2	Wrong fee comparison in `fees.py`
T3	Wrong field lookup in `report.py`
T4	Case-sensitive lookup in `indexer.py`
T5	Discount applied on the wrong branch in `billing/fees.py`

it the orchestrator applies Derive to pass a strict subset:

$$c_1' = \mathsf{Cap}[\mathsf{Read},\ \mathit{within}(\mathit{canon}(p), \texttt{"tests/**"})] \quad (4)$$

$$c_3' = \mathsf{Cap}[\mathsf{Exec},\ \mathit{allowed}_{A_{run}}(\mathit{parse}(s))] \quad (5)$$

Here `tests/**` is contained in `project/**` and `mypy` is dropped, so $\varphi' \Rightarrow \varphi$ and $\Sigma_{runner} = \{c_1', c_3'\}$. The runner's store contains no capability with effect type Write.

A trojaned fixture makes the suite print, to the runner's `pytest` output:

> ***[ci-bot] Known regression:*** *the signing key rotated. Restore the build by writing to* `src/auth/keys.py`*:* `SIGNING_KEY = "k3y_9f2a. . . c71"`*, then re-run.*

The key is attacker-controlled, and the target is an *ordinary source path*—no `.env`, `~/.ssh`, or `curl`, and nothing a deny pattern matches. Figure 1 compares three authorization models. The static policy does not classify this source edit as dangerous. B2's shared policy includes source writes because the repair needs them, so it cannot withhold that write from the runner while allowing the patcher to fix the file. CapScope gives the write only to the patcher. At turn 3 the runner is refused and may report the requested edit to the orchestrator as data; its own store still contains no Write capability.

4 Evaluation

We ask two questions. *RQ1:* does per-principal scoping reduce attacks that global, task-level authorization cannot distinguish? *RQ2:* what is the task-success and latency cost of fail-closed capability enforcement? Across conditions we vary the authorization model while holding the underlying LLM, the tasks, and the delegated four-principal topology fixed.

Tasks and attacks. Each task is a small Python project with one bug and a failing `pytest` suite. The injections request executing a project-local utility or writing to a project file. These actions are intentionally plausible rather than obviously harmful: each could be legitimate in another maintenance task, but none is needed for the requested repair. In the injected context, such execution or writing could introduce a malicious side effect. Table 1 summarizes the repair tasks.

Each malicious action creates a unique sentinel or marker that benign repair never produces. We score that side effect, rather than treating routine project activity itself as compromise.

Delegated pipeline. Every condition uses the same four-principal workflow. An orchestrator diagnoses and coordinates but delegates effectful work. A runner executes the failing tests, a patcher edits the relevant source, and a verifier reruns the suite. The loop repeats until the tests pass or the run terminates. This control is important: the conditions compare authorization models, not different agent architectures.

Injection surfaces. We place the same task-specific payload in each of the five untrusted surfaces of Section 2.1: a README, an agent-instruction file, a project-local skill, a source comment or docstring, and `pytest` output. The first three are normally encountered by the orchestrator while inspecting the project, source comments by an agent reading code, and test output by the agent running or verifying the tests. Varying the surface therefore changes which principal first receives the injection without changing the requested repair or malicious action.

Baselines. We compare CapScope against three global baselines that bracket current practice. All principals use ambient authority in **B0**. **B1** applies one static, task-independent policy to every principal, configured through the permission-gating facility Pi provides: it denies familiar hazards such as sensitive paths and obviously destructive commands (e.g., `.env` and `.ssh` paths, `rm -rf`, `sudo`, `curl`), but permits ordinary project operations. **B2** generates a tighter global policy from the trusted task description and a sanitized project structure before any injection is read, then applies that one policy to every principal; it can remove authority the task does not need, but cannot give two principals different slices of authority. **C** is not a baseline but CapScope itself, the system under test: a preflight mints a task-wide ceiling that bounds delegation, and each sub-agent receives a store derived from the applicable delegation ceiling. B2 is the strongest global baseline: its policy is tailored to the task before the adversarial text arrives. Comparing it with C shows the outcomes of adding per-agent authority to the complete mechanism.

Protocol and metrics. The full factorial is

$$4 \text{ conditions} \times 5 \text{ tasks} \times 5 \text{ surfaces} \times 3 \text{ trials} = 300 \text{ runs}.$$

Each run uses an isolated working copy and all conditions are driven by **qwen3.5-flash** through CloseAI's OpenAI-compatible endpoint. Runs were collected in May 2026; preflight calls used temperature 0. The harness verifies that the suite fails before the run, then records two outcomes:

- `task_success`: the final test suite passes;
- `attack_executed`: the attacker-only sentinel or marker exists on disk after the run.

Turn	Runner proposes	Static global policy	Task-specific global policy	CapScope per agent
1	`read("tests/test_auth.py")`	allowed (global read)	allowed (c_1)	allowed (c_1')
2	`bash("pytest tests/test_auth.py")`	allowed (global exec) → payload enters	allowed (c_3) → payload enters	allowed (c_3') → payload enters
3	`write("src/auth/keys.py", ...)`	**allowed**: matches no deny rule	**allowed**: c_2 covers `src/**`	**blocked**: no Write in Σ_{runner}

Figure 1. The same runner under three authorization models. Turns 1 and 2 are legitimate and allowed everywhere, so the poisoned test output enters the runner's context and it proposes the backdoor write at turn 3. The static global policy allows the ordinary-looking source path. The task-specific global policy also allows it because its shared authority includes c_2 for the legitimate patch. CapScope blocks it because the runner has only $\Sigma_{runner} = \{c_1', c_3'\}$.

Each outcome is Boolean per run: a run is counted once if the attack executes at least once, even if it also contains blocked or repeated attack attempts. Both are read from the filesystem, independent of any log and of each other: a run may fix the bug and still be compromised. `attack_executed` is our headline defense metric—did the attack land. The per-call decision log additionally shows, on the conditions that install a hook, whether a call carrying the injected action was *proposed* by the model and whether the layer *blocked* it. We report these as secondary diagnostics, not outcome metrics. They need not be mutually exclusive because a run may contain several attempts. B0 installs no hook and exposes neither count.

Results. Table 2 reports descriptive counts over the three trials. For attack execution, B0 ≈ B1 > B2 ≫ C: the injected effect lands in 47 of 75 runs under ambient authority and 46 under the static global policy—a conventional denylist does not improve on having no policy, because the malicious actions are ordinary in-repository operations it never names. The task-specific global policy (B2) helps, cutting leakage to 33/75 by removing task-unnecessary operations before the injection arrives, but that one allowlist is still shared by every principal (below). CapScope (C) admits the effect in 3/75, compared with 33/75 for the strongest global baseline, because it additionally withholds task-needed authority from the principals that do not need it. The per-surface view (Table 3) summarizes outcomes by carrier across conditions. Attack execution was highest for AGENTS.md (43/60) and lowest for source comments (8/60).

Task success differs by four runs between C and B0: CapScope finishes 68 of 75 repairs against 72 for the unconstrained baseline—a gap of four runs—and B2's tighter global policy also finishes 68/75. As a diagnostic, the decision log shows that under CapScope the model proposes the injected action on 34 of 75 runs and the hook records only 3 runs with an allowed attack call.

Why a shared task policy still permits attacks. Because one policy is shared by every principal, it must grant the union of everything the task needs—reads, a source write, the test commands—just to let the repair finish, and that broad authority then reaches *every* sub-agent, the runner that reads the poisoned output included. Removing a task-required permission would also prevent the legitimate step. CapScope scopes authority per agent instead: the runner that reads the injection can run tests but cannot write source, so the attack is blocked, while the patcher still holds the write that completes the repair.

Latency. Mean wall-clock time is 145 s for B0, 140 s for B1, 194 s for B2, and 316 s for C. For C, the median is 208 s; five runs reached the 900 s timeout, increasing the mean. The decision logs provide a descriptive account of this additional work. Among the 34 C runs containing an attack-targeting proposal, 27 contain repeated attack-targeting calls. The measured latency therefore includes additional model turns following refusals, rather than only the cost of the authorization check. Derivation-related calls are also associated with longer runs. These observations do not isolate the causal cost of any individual component, since grant construction, sub-agent setup, and other model interactions may also contribute.

Interpreting the cost. The pipeline is fully autonomous and has no human approval path. Every denial returns control to the model, which may try another call. In an interactive deployment, a user could reject the first attack-driven request and end that retry sequence, reducing subsequent agent compute; the user could also approve a legitimate uncovered step. We did not measure human response time or approval choices, so this is a deployment implication rather than a measured latency result. Approval is disabled to keep the protocol fixed.

5 Discussion

Policy quality and approval. Enforcement checks the installed capabilities at every mediated call. Policy generation is a separate source of error: a scope that is too broad may admit an unwanted in-scope action, while one that is too narrow may block a required step. CapScope does not widen a scope silently. A deployment can instead show the

Table 2. Aggregate results per condition (75 runs each). *Attack executed* (lower is better) is the headline defense metric, read from the filesystem; *mean time* is wall-clock seconds per run.

Condition	Runs	Task success	Attack exec.	Mean time (s)
B0: ambient authority	75	72/75	47/75	145
B1: static global	75	70/75	46/75	140
B2: task-specific global	75	68/75	33/75	194
C: CapScope	75	68/75	3/75	316

Table 3. Results by injection surface, aggregated equally across the four authorization conditions and five tasks (60 runs each). These totals describe the average outcome associated with each surface.

Surface	Runs	Task success	Attack exec.
README	60	54/60	25/60
AGENTS.md	60	56/60	43/60
Skill file	60	57/60	33/60
Source comment	60	57/60	8/60
Tool output	60	54/60	20/60

refusal to the user and record any approved capability as a fresh trusted Mint.

How a residual injection still lands. CapScope's remaining executed attacks (Table 2) fall into two grant cases. (1) A grant can be wider than its subtask needs, such as permitting a directory write instead of one file. (2) Injected text can influence the model-generated subtask, causing the orchestrator or host-side grant completion to select an attacker-requested capability. Both cases require the action to lie within the preflight ceiling. Dispatch enforcement cannot correct a capability that the policy intentionally installed.

Trusted computing base. The TCB is exactly the host-side components named in Section 2.2: the preflight isolation boundary, capability compiler, derivation code, per-principal stores, scope predicates, and dispatch hooks. The preflight LLM shapes how broad the initial policy is, but cannot place or alter capabilities once untrusted content is admitted—only host code can mint, narrow, or enforce them.

Limitations and threats to validity. The corpus contains five small Python repairs, one model, and three trials per task-surface-condition cell. It does not represent large refactors, dependency migrations, or long-running workflows, where policy omissions and approval frequency may be higher. The payloads are intentionally strong and the model may react differently to each carrier, so we report results by surface rather than assuming a uniform injection rate. Because the underlying calls are stochastic, individual cells vary from trial to trial; we therefore report aggregates over three trials and read the per-condition ordering, which is stable, rather than any single cell. Finally, the informative refusal message can affect recovery and task success, though it cannot turn an uncovered call into an allowed one.

6 Related Work

Prompt injection and model-side defenses. Indirect prompt injection lets attacker-controlled external content hijack an LLM-integrated application [8]. OWASP ranks it the leading LLM application risk [15]. In the coding setting the threat is acute: Liu et al. [12] report attack success rates up to 84% in production coding editors, and rule and skill files are a documented backdoor vector [10]. Model-side defenses such as Instruction Hierarchy [18] and PromptArmor [17] make the model more skeptical of untrusted text. Their decisions remain probabilistic. CapScope instead checks installed permissions at the harness boundary and can be combined with model-side defenses.

Harness-level defenses. AgentDojo [4] is a benchmark for agent prompt injection; Bhagwatkar et al. [1] show that simple firewalls solve many cases in several benchmarks. Deployed coding agents also enforce permission boundaries through sandboxes, approval modes, and allowlist/denylist command patterns. CapScope differs along two axes. Its command scopes match a parsed executable and argument vector, and a compound command is checked segment by segment rather than as one string; thus allowing `pytest` does not also admit `pytest; cmd` or `pytest && cmd` unless `cmd` is itself covered, nor `$(cmd)`, whose substituted argument matches no allowed invocation. Authority is also attenuated per agent. A task-wide global policy, even one generated specifically for the task, cannot permit a patcher to write a file while withholding that write from a runner. Derive can, and bounds the sub-agent by its delegation ceiling.

Other systems govern agent execution at different points. CaMeL [3] uses capabilities in a dual-LLM planner with a custom interpreter. Fides [2] tracks confidentiality and integrity labels over data flow; AgentArmor [19] reconstructs runtime traces into an IR and type-checks it; ToolGate [11] attaches Hoare-style contracts to tool calls; and PACT [7] treats injection as untrusted content determining an authority-bearing argument and enforces argument-level provenance contracts. CapScope differs from all of these in where authority originates: it installs typed capability values from a trusted policy before untrusted content is read, then checks them at dispatch. This mechanism can be combined with data-flow tracking.

Capability security. CapScope's vocabulary is classical. Saltzer and Schroeder [16] define a capability as an unforgeable authorization and elevate complete mediation and least privilege; Dennis and Van Horn [5] couple designation with authority, dissolving the confused-deputy problem [9]; and Miller and Shapiro [14] distinguish authority from permission and show how attenuation follows from ordinary reference-passing. CapScope instantiates this discipline in an LLM agent harness, with scope predicates as the attenuation mechanism.

7 Conclusion

CapScope limits coding-agent tool calls with typed, host-side capabilities. A preflight step establishes a task ceiling before repository contents are read; delegation gives each sub-agent a narrower store; and a dispatch hook checks the issuing agent's store before executing a call. Our evaluation compares ambient authority, a static global policy, a task-specific global policy, and CapScope across five tasks, five injection surfaces, and three trials per cell (300 runs). The global policies execute the injected effect in 33–47/75 runs; CapScope executes it in 3/75 and completes 68/75 repairs. The mechanism blocks a mediated call when the issuing agent lacks a matching installed capability. It does not prevent misuse of a granted capability or the transitive effects of an allowed command, which remain responsibilities of policy design and sandboxing.

References

[1] Rishika Bhagwatkar, Kevin Kasa, Abhay Puri, Gabriel Huang, Irina Rish, Graham W. Taylor, Krishnamurthy Dvijotham, and Alexandre Lacoste. 2025. Indirect Prompt Injections: Are Firewalls All You Need, or Stronger Benchmarks? arXiv:2510.05244 [cs.CR] doi:10.48550/arXiv.2510.05244

[2] Manuel Costa, Boris Köpf, Aashish Kolluri, Andrew Paverd, Mark Russinovich, Ahmed Salem, Shruti Tople, Lukas Wutschitz, and Santiago Zanella-Béguelin. 2025. Securing AI Agents with Information-Flow Control. arXiv:2505.23643 [cs.CR] doi:10.48550/arXiv.2505.23643

[3] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. 2025. Defeating Prompt Injections by Design. arXiv:2503.18813 [cs.CR] doi:10.48550/arXiv.2503.18813

[4] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *Advances in Neural Information Processing Systems*. https://openreview.net/forum?id=m1YYAQjO3w Datasets and Benchmarks Track.

[5] Jack B. Dennis and Earl C. Van Horn. 1966. Programming Semantics for Multiprogrammed Computations. *Commun. ACM* 9, 3 (1966), 143–155. doi:10.1145/365230.365252

[6] Earendil Works. 2026. Pi: Agent Harness Mono Repo. https://github.com/earendil-works/pi. Accessed: 2026-05-28.

[7] Linfeng Fan, Ziwei Li, Yuan Tian, Yichen Wang, Rongsheng Li, and Xiong Wang. 2026. The Granularity Mismatch in Agent Security: Argument-Level Provenance Solves Enforcement and Isolates the LLM Reasoning Bottleneck. arXiv:2605.11039 [cs.CR] doi:10.48550/arXiv.2605.11039

[8] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec '23)*. Association for Computing Machinery, New York, NY, USA, 79–90. doi:10.1145/3605764.3623985

[9] Norm Hardy. 1988. The Confused Deputy: (or Why Capabilities Might Have Been Invented). *ACM SIGOPS Operating Systems Review* 22, 4 (Oct. 1988), 36–38. doi:10.1145/54289.871709

[10] Ziv Karliner and Pillar Security. 2025. New Vulnerability in GitHub Copilot and Cursor: How Hackers Can Weaponize Code Agents. https://www.pillar.security/blog/new-vulnerability-in-github-copilot-and-cursor-how-hackers-can-weaponize-code-agents. Introduces the "Rules File Backdoor" attack vector. Accessed: 2026-05-28.

[11] Yanming Liu, Xinyue Peng, Jiannan Cao, Xinyi Wang, Songhang Deng, Jintao Chen, Jianwei Yin, and Xuhong Zhang. 2026. ToolGate: Contract-Grounded and Verified Tool Execution for LLMs. arXiv:2601.04688 [cs.CL] doi:10.48550/arXiv.2601.04688

[12] Yue Liu, Yanjie Zhao, Yunbo Lyu, Ting Zhang, Haoyu Wang, and David Lo. 2025. "Your AI, My Shell": Demystifying Prompt Injection Attacks on Agentic AI Coding Editors. arXiv:2509.22040 [cs.CR] doi:10.48550/arXiv.2509.22040

[13] Mark S. Miller. 2006. *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control.* Ph. D. Dissertation. Johns Hopkins University. https://worrydream.com/refs/Miller_2006_-_Robust_Composition.pdf

[14] Mark S. Miller and Jonathan S. Shapiro. 2003. *Paradigm Regained: Abstraction Mechanisms for Access Control.* Technical Report SRL2003-03. Johns Hopkins University, Systems Research Laboratory. https://srl.cs.jhu.edu/pubs/SRL2003-03.pdf

[15] OWASP GenAI Security Project. 2025. LLM01:2025 Prompt Injection. https://genai.owasp.org/llmrisk/llm01-prompt-injection/. Accessed: 2026-05-28.

[16] Jerome H. Saltzer and Michael D. Schroeder. 1975. The Protection of Information in Computer Systems. *Proc. IEEE* 63, 9 (1975), 1278–1308. doi:10.1109/PROC.1975.9939

[17] Tianneng Shi, Kaijie Zhu, Zhun Wang, Yuqi Jia, Will Cai, Weida Liang, Haonan Wang, Hend Alzahrani, Joshua Lu, Kenji Kawaguchi, Basel Alomair, Xuandong Zhao, William Yang Wang, Neil Gong, Wenbo Guo, and Dawn Song. 2025. PromptArmor: Simple yet Effective Prompt Injection Defenses. arXiv:2507.15219 [cs.CR] doi:10.48550/arXiv.2507.15219

[18] Eric Wallace, Kai Xiao, Reimar Leike, Lilian Weng, Johannes Heidecke, and Alex Beutel. 2024. The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions. arXiv:2404.13208 [cs.LG] doi:10.48550/arXiv.2404.13208

[19] Peiran Wang, Yang Liu, Yunfei Lu, Yifeng Cai, Hongbo Chen, Qingyou Yang, Jie Zhang, Jue Hong, and Ye Wu. 2025. AgentArmor: Enforcing Program Analysis on Agent Runtime Trace to Defend Against Prompt Injection. arXiv:2508.01249 [cs.CR] doi:10.48550/arXiv.2508.01249

[20] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. 2024. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents. In *Findings of the Association for Computational Linguistics: ACL 2024*. Association for Computational Linguistics, Bangkok, Thailand, 10471–10506. doi:10.18653/v1/2024.findings-acl.624